#8

CS 0447
Introduction to
Computer Programming

Bitwise Operations and Bitfields

Luís Oliveira

Original slides by: Jarrett Bi
Modified with bits from: Br

Fall 2020

Announcements

Logical, right?

```
class Main {
        public static void main(String[] args) {
                boolean a = true;
                boolean b = true;
                System.out.println(a||b);
        }
}
```

- **What is the output?**

true

```
class Main {
        public static void main(String[] args) {
                int a = 4;
                int b = 1;
                System.out.println( a|b );
        }
}
```

- **What is the output?**

5

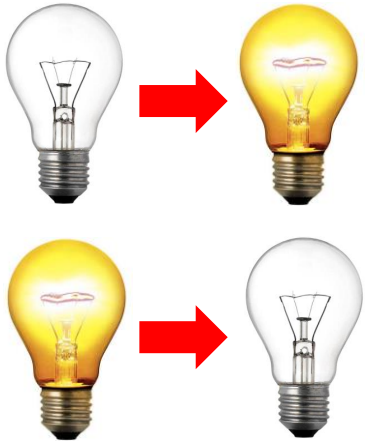
What are "bitwise" operations?

- The "numbers" we use on computers aren't *really* numbers right?
- It's often useful to treat them instead as **a pattern of bits**
- **Bitwise operations** treat a value as a pattern of bits



The simplest operation: NOT (logical negation)

- If the light is off, turn it on
- If the light is on, turn it off

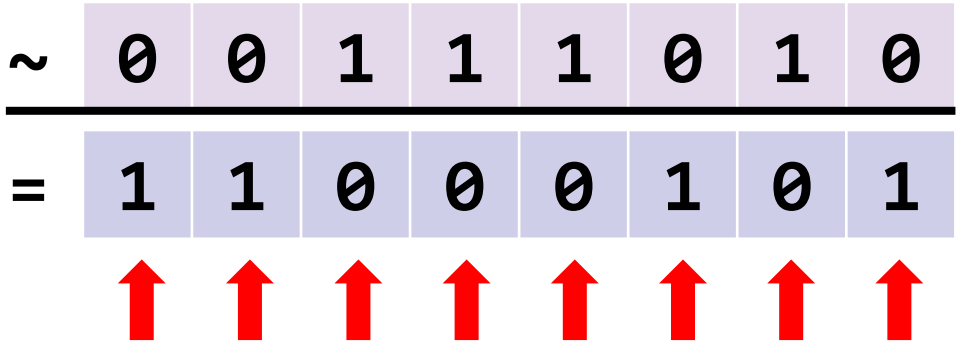


A	Q
0	1
1	0

- We can summarize this in a **truth table**
- We write NOT as **~A**, or **¬A**, or $\overline{\mathbf{A}}$

Applying NOT to a whole bunch of bits

- if we use the **not** instruction (or **~** in C/Java), this is what happens:



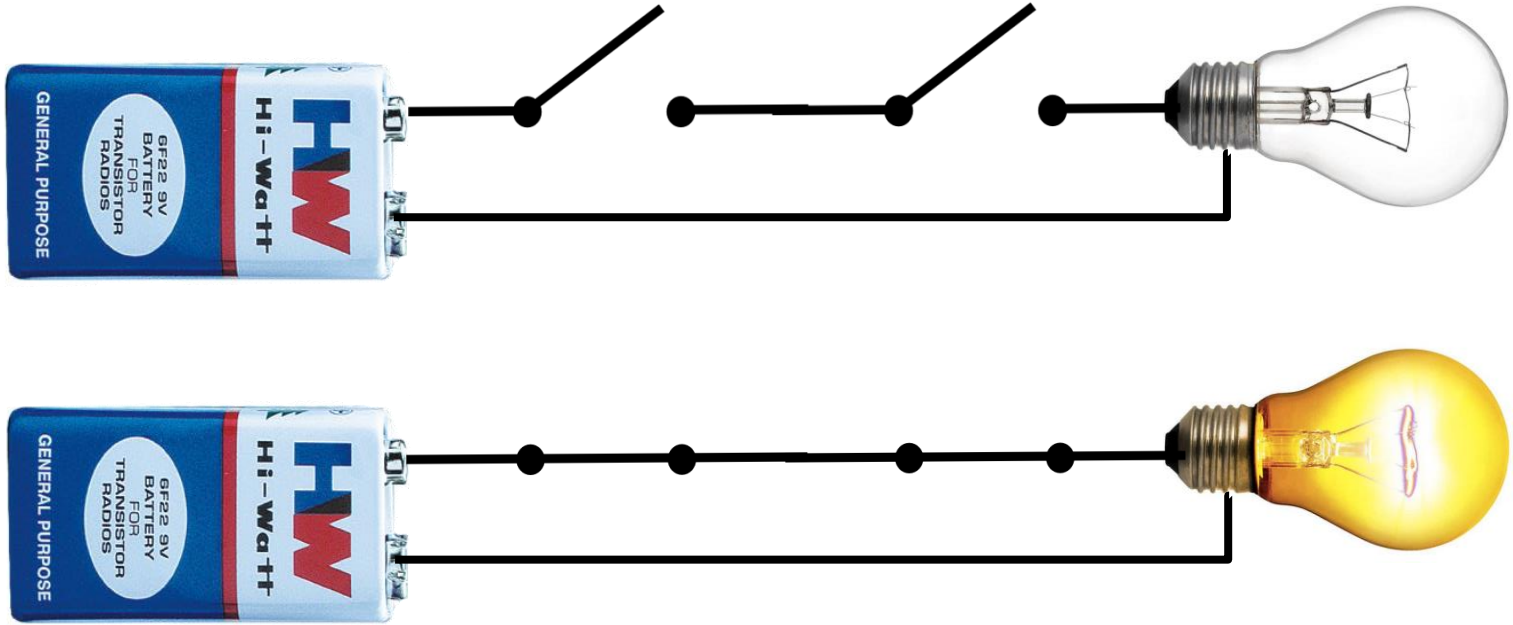
we did 8 **independent** NOT operations

that's it it's super simple

only 8 bits shown cause 32 bits on a slide is too much

Let's add some switches

- There are two switches in a row connecting the light to the battery
- **How do we make it light up?**



AND (Logical product)

- AND is a **binary (two-operand) operation**
- it can be written a number of ways:
  - **A&B A∧B A·B AB**
- if we use the **and** instruction (or **&** in C/Java):

	1	1	1	1	0	0	0	0
&	0	0	1	1	1	0	1	0
=	0	0	1	1	0	0	0	0

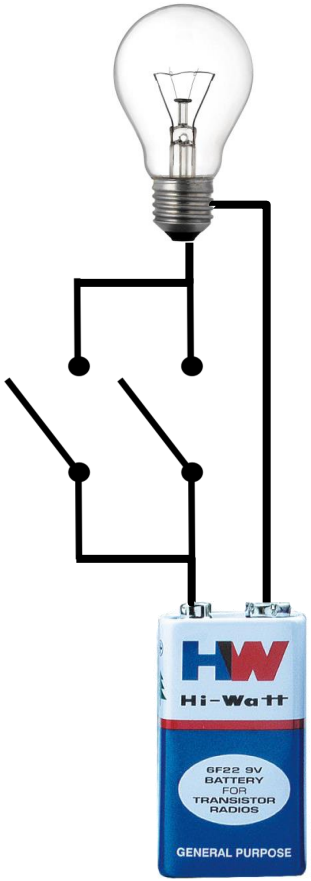


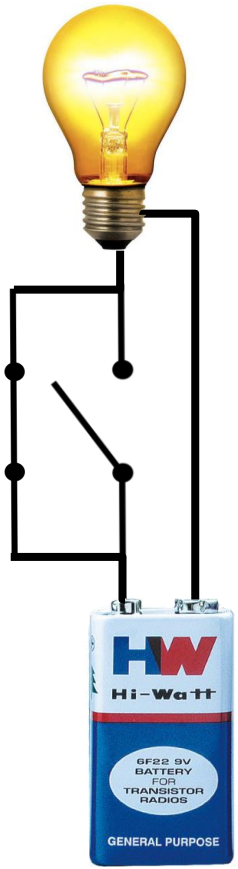
we did 8 **independent** AND operations

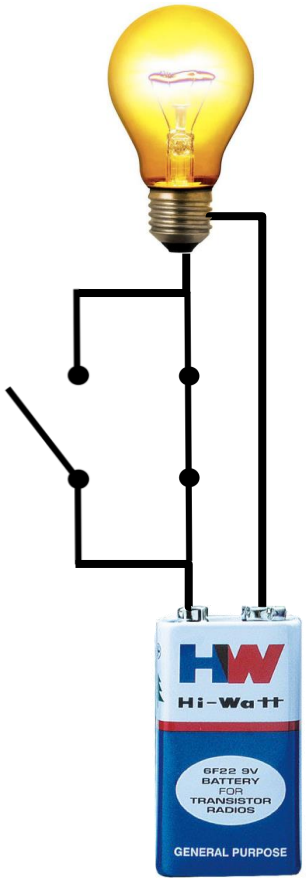
A	B	Q
0	0	0
0	1	0
1	0	0
1	1	1

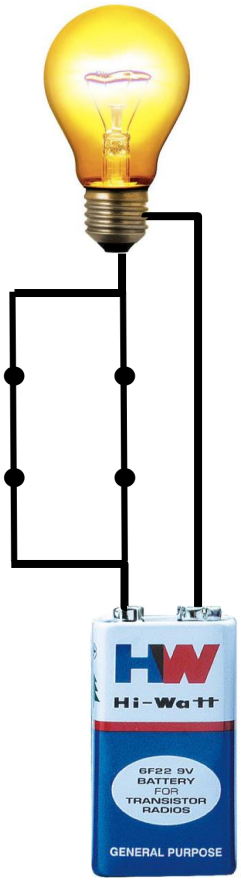
"Switching" things up

- NOW how can we make it light up?









OR (Logical sum...?)

- we might say **"and/or"** in English
- it can be written a number of ways:
  - **A|B AVB A+B**
- if we use the **or** instruction (or **|** in C/Java):

	1	1	1	1	0	0	0	0
|	0	0	1	1	1	0	1	0
=	1	1	1	1	1	0	1	0

We did 8 **independent** OR operations.

A	B	Q
0	0	0
0	1	1
1	0	1
1	1	1

lui, ori...

- If I write `li t0, 0xDEADBEEF` in MIPS, the assembler turns it into:

```
lui at, 0xDEAD
ori t0, at, 0xBEEF
```



- **at** is used by the assembler, sooooo...
- The reason it splits it up is that **there's only enough space in each instruction to fit half of 0xDEADBEEF**
  - As we've seen in the lab, **each immediate is 16 bits long**
  - We'll learn about instruction encoding later
- What the heck are these instructions *doing* tho

By your powers combined…

- **lui** means **load upper immediate.** it puts the immediate value into the **upper 16 bits of the register**, and zeroes out the rest

```
lui at, 0xDEAD
```

- then, **ori** does logical OR of **at** and its **zero-extended** immediate

```
ori t0, at, 0xBEEF
```

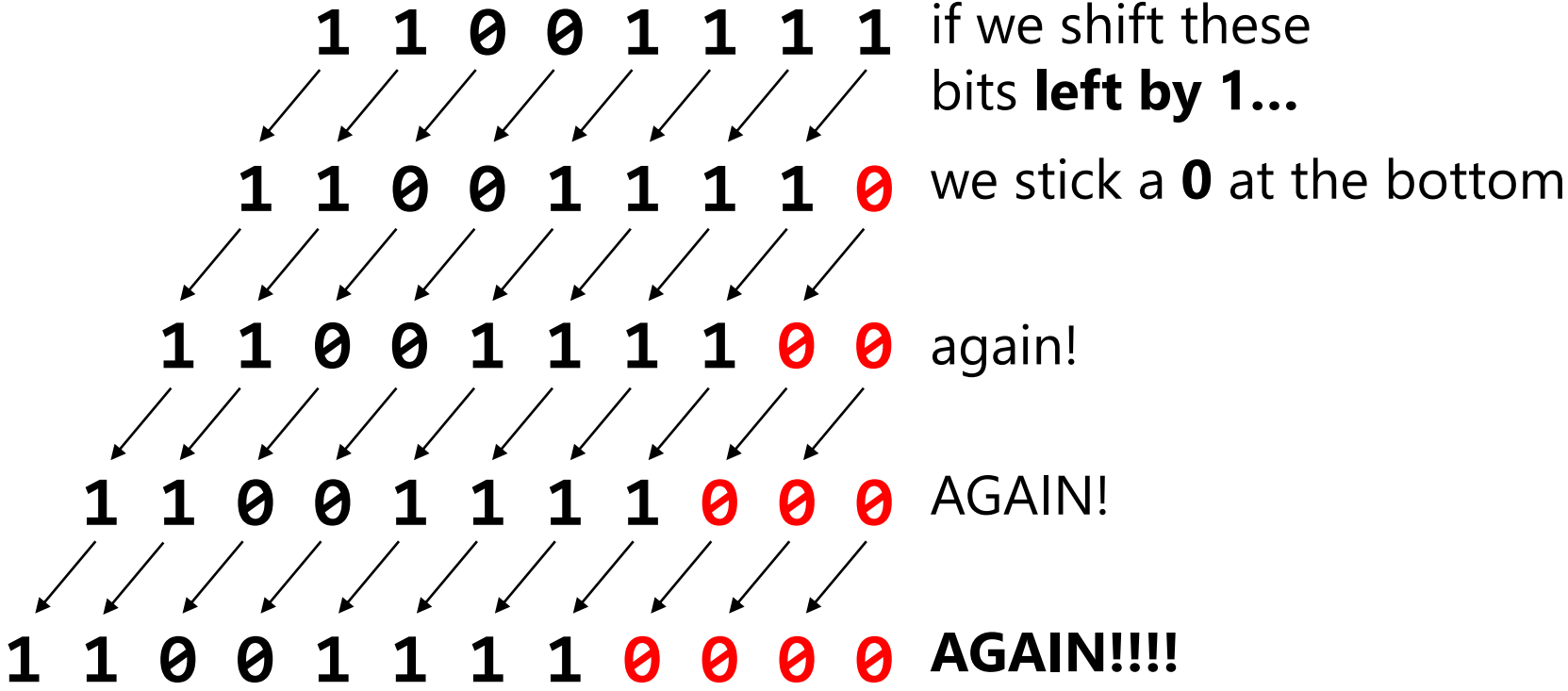
```
  1101111010101101000000000000000
| 0000000000000000101111011101111
-------------------------------
  1101111010101100101111011101111

  D    E    A    D    B    E    E    F
```

Bit shifting

Bit shifting

- besides AND, OR, and NOT, we can **move bits around,** too.



Left-shifting in C/Java and MIPS (animated)

- C and Java use the << operator for left shift

```
B = A << 4; // B = A shifted left 4 bits
```

- MIPS has the **sll** (**S**hift **L**eft **L**ogical) instruction

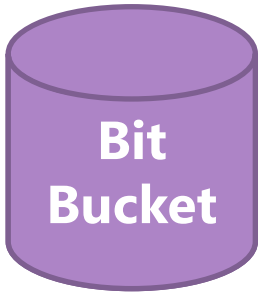
```
sll t2, t0, 4 # t2 = t0 << 4
```

- MIPS has the **sllv** (**S**hift **L**eft **L**ogical **V**ariable) instruction
  - No, registers are not variables!

```
sllv t2, t0, t1 # t2 = t0 << t1
```

- if the bottom 4 bits of the result are now 0s...
  - ...what happened to the *top* 4 bits?

0011 0000 0000 1111 1100 1101 1100 1111



the bit bucket is not a real place

it's a programmer joke ok

- we can **shift right, too**

```
0 0 1 1 0 0 0 0 0 0 0 0 1 1 1 1 1 1 0 0 1 1 0 1 1 1 0 0 1 1 1 1
0 0 0 1 1 0 0 0 0 0 0 0 0 1 1 1 1 1 1 0 0 1 1 0 1 1 1 0 0 1 1 1
0 0 0 0 1 1 0 0 0 0 0 0 0 0 1 1 1 1 1 1 0 0 1 1 0 1 1 1 0 0 1 1
0 0 0 0 0 1 1 0 0 0 0 0 0 0 0 1 1 1 1 1 1 0 0 1 1 0 1 1 1 0 0 1
0 0 0 0 0 0 1 1 0 0 0 0 0 0 0 0 1 1 1 1 1 1 0 0 1 1 0 1 1 1 0 0
```

  - C/Java use >>, MIPS uses **srl** (**S**hift **R**ight **L**ogical)

Numbers they are a-changing

- let's start with a value like 5 and shift left and see what happens

Binary	Decimal
101	5
1010	10
10100	20
101000	40
1010000	80

why is this happening

well uh... what if I gave you

49018853

how do you multiply that by 10?

by 100?

by 100000?

something **very similar** is happening here

$a << n == a * 2^n$

- **shifting left by *n* is the same as multiplying by 2^n**
  - you probably learned this as "moving the decimal point"
    - and moving the decimal point *right* is like shifting the digits *left*
- **shifting is fast and easy on most CPUs**
  - way faster than multiplication in any case
- hey… if shifting *left* is the same as multiplying…

a >> n == a / 2^n, ish

- You got it
- **Shifting right by n is like dividing by 2^n**
  - ***sort of.***
- What's 5_{10} (0101_2) shifted right by 1?
  - 10_2, which is 2…
    - It's like doing **integer** (or **flooring**) division
    - Which is a fancy way of saying we round to the smallest number

- What if the number is signed?
- What's -3_{10} (**next class**: 1101_2) shifted right by 1?
  - 0110_2, which is 6_{10}???
    - Ohhhhh

- We can do **sign-extension on shifts**, too!!

```
1 1 1 1 0 0 0 0 0 0 0 0 1 1 1 1 1 1 0 0 1 1 0 1 1 1 0 0 1 1 1 1
1 1 1 1 1 0 0 0 0 0 0 0 0 1 1 1 1 1 1 0 0 1 1 0 1 1 1 0 0 1 1 1
1 1 1 1 1 1 0 0 0 0 0 0 0 0 1 1 1 1 1 1 0 0 1 1 0 1 1 1 0 0 1 1
1 1 1 1 1 1 1 0 0 0 0 0 0 0 0 1 1 1 1 1 1 0 0 1 1 0 1 1 1 0 0 1
1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0 1 1 1 1 1 1 0 0 1 1 0 1 1 1 0 0
```

  - MIPS uses **sra** (**S**hift **R**ight **A**rithmetic)

  - What's -3_{10} (1101_2) shifted right by 1?
    - 1110_2, which is -2_{10}
      - **Why is this -2????? next class**

MIPS ISA: Bitwise operations

- Bitwise logical operators

Instruction	Meaning
`not a, b`	`a = ~b`
`or a, b, c`	`a = b|c`
`ori a, b, imm`	`a = b|imm`
`and a, b, c`	`a = b&c`
`andi a, b, imm`	`a = b&imm`

MIPS ISA: Bitwise shifts

- Bitwise logical operators

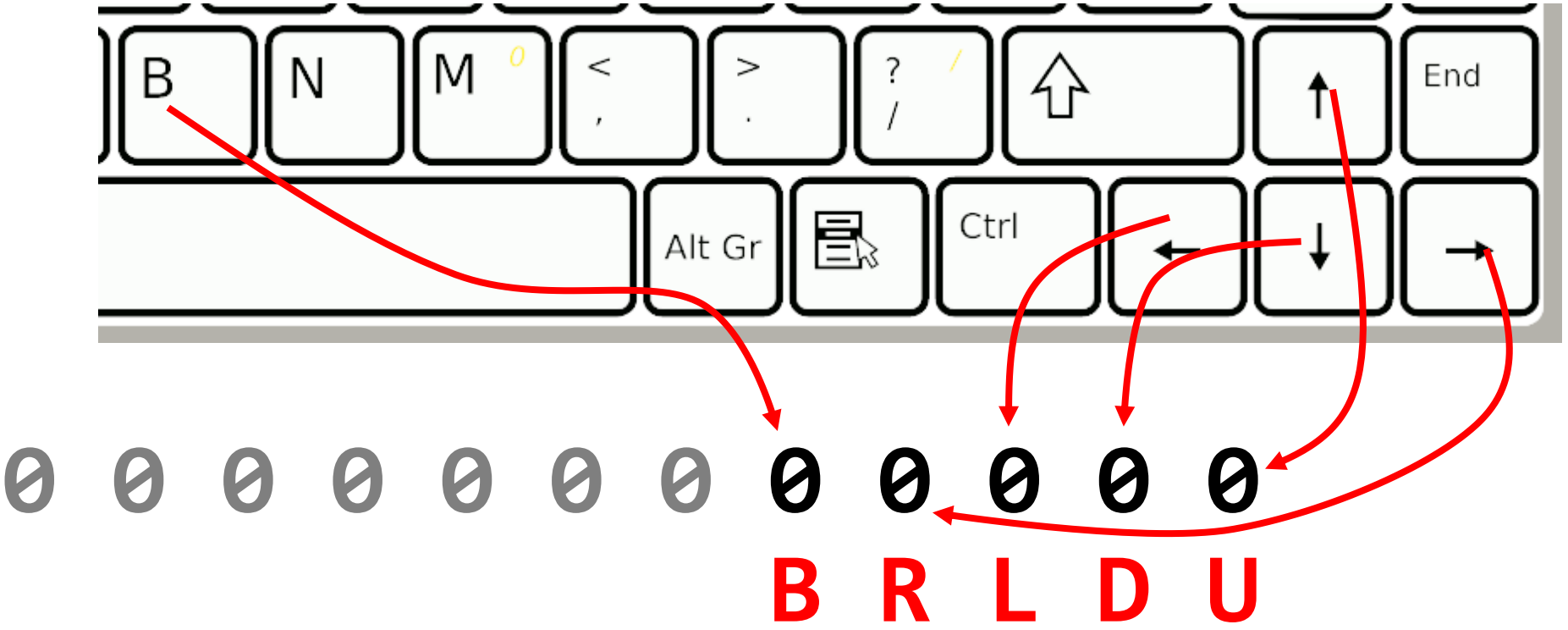
Instruction	Meaning
`sll a, b, imm`	`a = b<<imm`
`sllv a, b, c`	`a = b<<c`
`srl a, b, imm`	`a = b>>imm (zero extension)`
`srlv a, b, c`	`a = b>>c (zero extension)`
`sra a, b, imm`	`a = b>>imm (sign extension)`
`srav a, b, c`	`a = b>>c (sign extension)`

Bitfields

clicky clicky

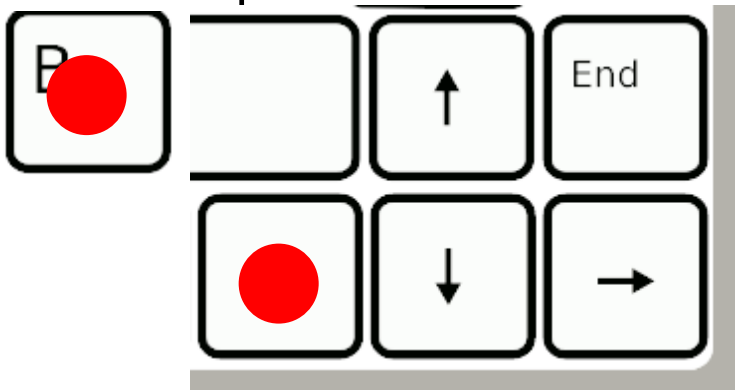
- In the LED Keypad plugin in MARS, input works like this:

`input_get_keys` returns a value in v0…



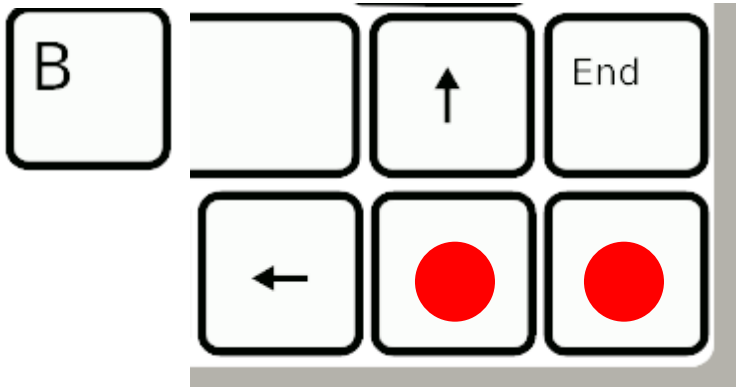
Why do we do this??

- It lets us cram several booleans into a **single** value!
- This technique is known as **bit flags**



1 0 1 0 0

B R L D U

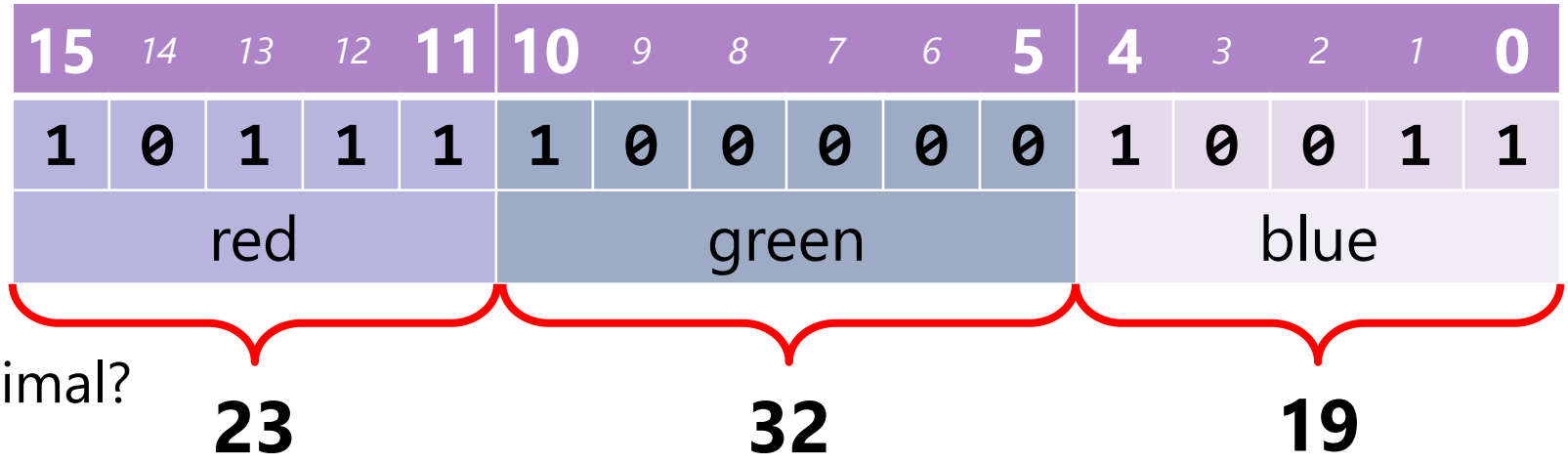


0 1 0 1 0

B R L D U

The masters of meaning

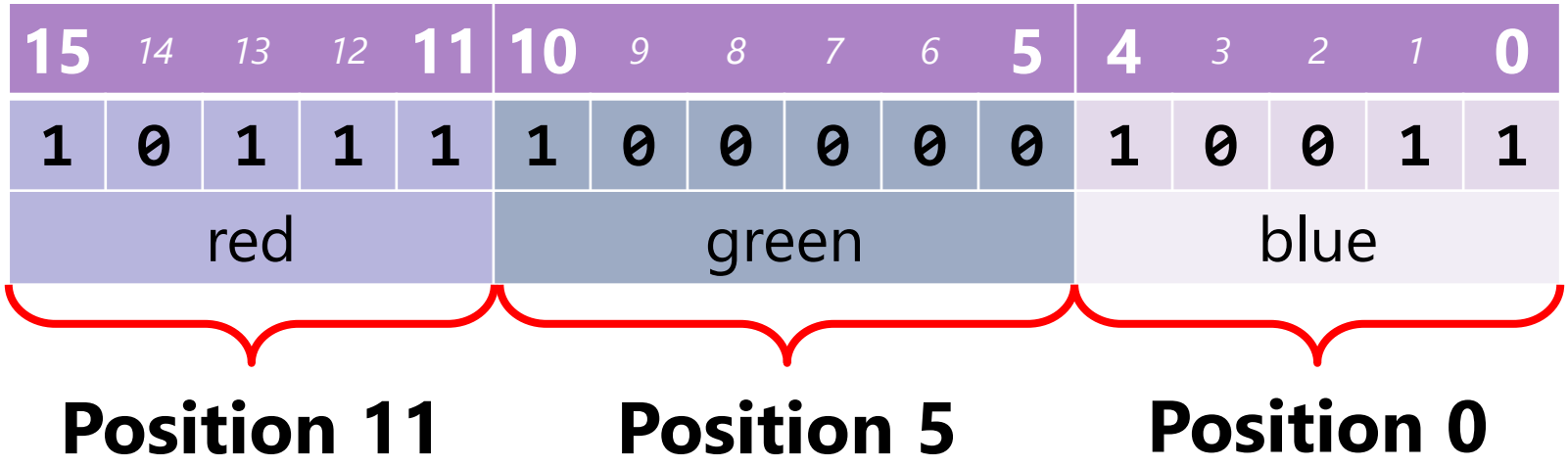
- well what if we wanted to store multiple ***integers*** in one value?



That's this color, in RGB565.

The masters of meaning

- This bitfield has 3 fields: red, green, and blue



Whyyyyy???

- It's **smaller**
  - really, that's it
  - but that's super important in a lot of cases
- **Smaller data…**
  - Takes up **less space** in memory
  - Takes up **less space** in *cache*
    - extremely important thing in modern CPUs that we talk about in 1541
  - Is **faster to move** between memory and the CPU
  - Is **faster to transfer** across the internet and other networks
  - It allows a MIPS instruction to contain references to multiple registers

I wanna turn the light on!!

- I have a sequence of 0s. I wanna turn one of them into a 1.
- what **bitwise operation** can I use to do that?



I wanna turn the light off!!

- I wanna turn one of the 1s into a 0.
- what **bitwise operation** can I use to do *that?*



Turning off the first three, leaving the others alone

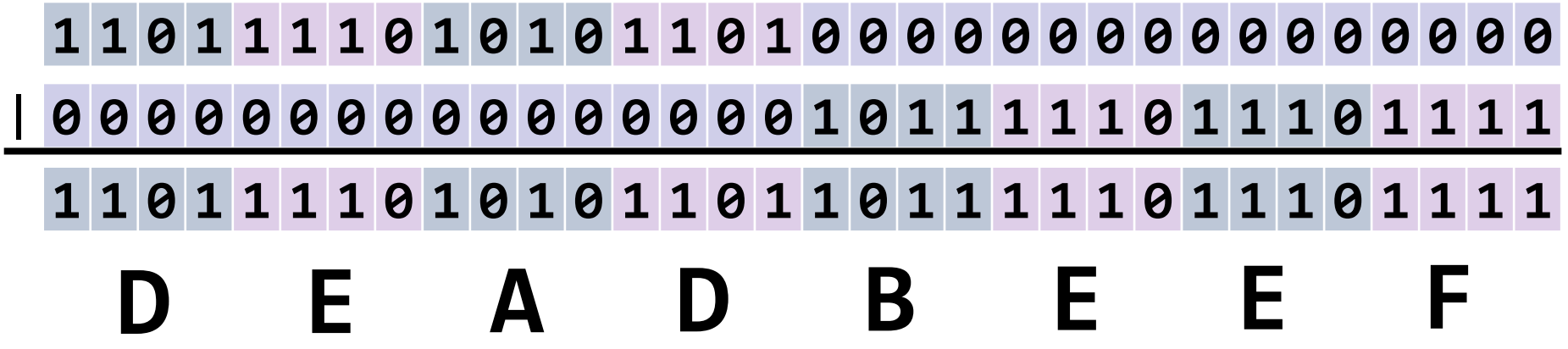
- more bits, but one of the same operations…



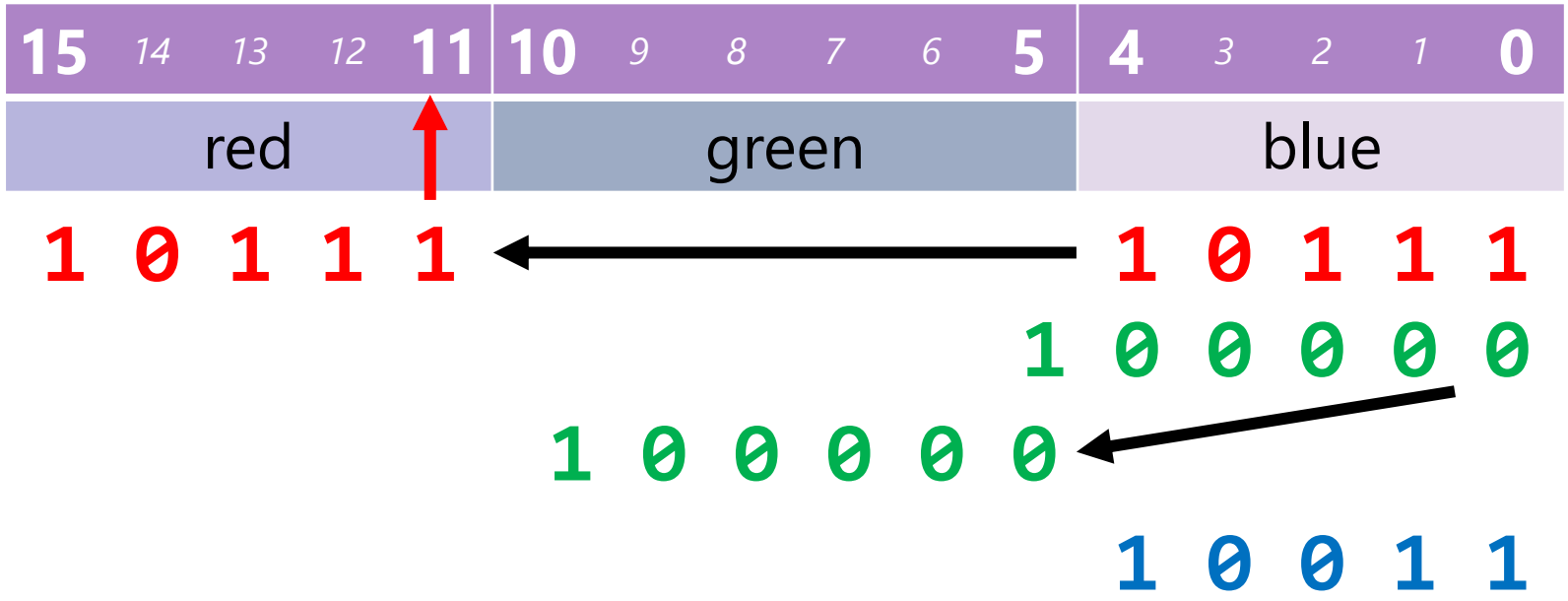
```
   1  0  1  0  1  1
?  0  0  0  1  1  1
   -----------------
   0  0  0  0  1  1
```

Remember this?

```
lui at, 0xDEAD
ori t0, at, 0xBEEF
```



How can we assemble on of these bitfields?



- hmm

```
  1 0 1 1 1   0 0 0 0 0 0 0 0 0 0 0

  0 0 0 0 0   1 0 0 0 0 0 0 0 0 0 0
  0 0 0 0 0   0 0 0 0 0 0 1 0 0 1 1
-------------------------------------
  1 0 1 1 1   1 0 0 0 0 0 1 0 0 1 1
```

Left-shifting and ORing

- if you have the **values of the fields**
- and you want to **put them together into a bitfield**
  - **shift** each value **left** to the correct bit position
  - **OR** the **shifted values** together
- for RGB565,
  - red is shifted left 11
  - green is shifted left 5
  - blue isn't shifted (shifted left 0…)

```
color = (red << 11) | (green << 5) | blue;
```

Going the other way

- let's go from the bitfield to **three separate values.**

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	1	1	0	0	0	0	0	1	0	0	1	1

let's say we somehow set all the non-red bits to 0.

1	0	1	1	1	0	0	0	0	0	0	0	0	0	0	0

what value is this?

it's not 23, that's for sure.

so **how do we fix that?**

It's the exact opposite

- we have to **shift right** to put the field at position 0

15	*14*	*13*	*12*	**11**	**10**	*9*	*8*	*7*	*6*	**5**	**4**	*3*	*2*	*1*	**0**
1	**0**	**1**	**1**	**1**	**1**	**0**	**0**	**0**	**0**	**0**	**1**	**0**	**0**	**1**	**1**

shift right by 11 and...

0	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**1**	**0**	**1**	**1**	**1**

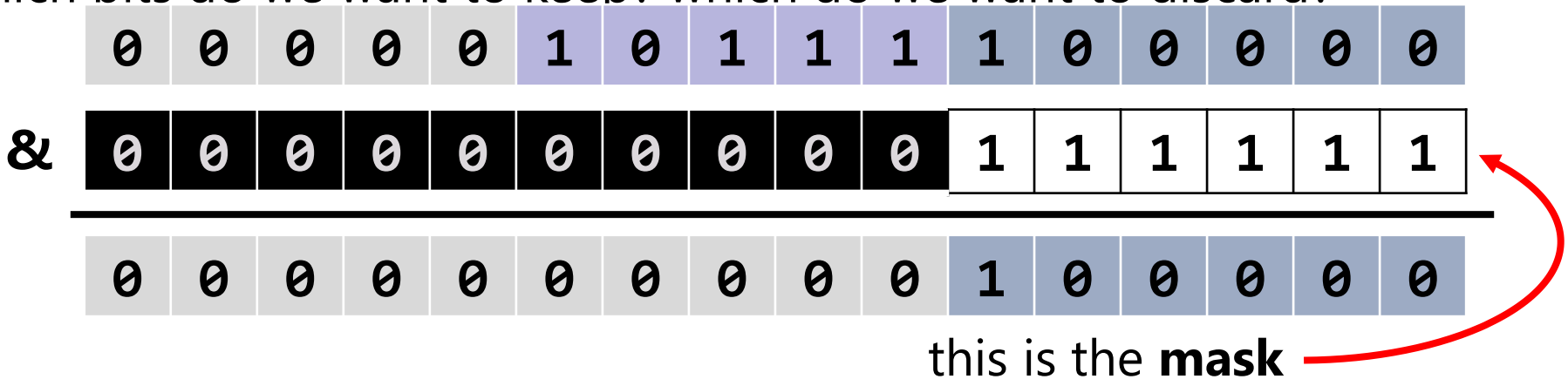
cool. what about green? shift right by...?

0	**0**	**0**	**0**	**0**	**1**	**0**	**1**	**1**	**1**	**1**	**0**	**0**	**0**	**0**	**0**

uh oh.

Masquerade

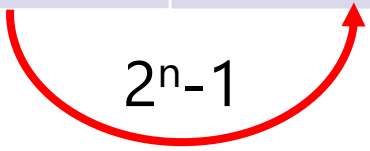
- we need to **get rid of (zero out)** the bits that we don't care about
- a **mask** is a **specially-constructed value** that has:
  - 1s in the bits that we want to keep
  - 0s in the bits that we want to discard
- which bits do we want to keep? which do we want to discard?



Coming up with the mask value

- if you want to mask a 3 bit value, the mask is $\mathbf{111_2}$
- if you want to mask a 4 bit value, the mask is $\mathbf{1111_2}$
- if you want to mask a 5 bit value, it's...?

Size(n)	Mask	2^n	Mask in decimal
3	111_2	8	7
4	1111_2	16	15
5	11111_2	32	31



Right-shifting and ANDing

- to **extract** one or more fields from a bitfield:
  - **shift** the bitfield **right** to put the desired field at bit position 0
  - **AND** that with **2^n-1**, where *n* is the number of bits in the field
- so for RGB565…
  - the red and blue masks are 2^5-1 = 31 (or **0x1F**)
  - the green mask is 2^6-1 = 63 (or **0x3F**)

```
red = (color >> 11) & 0x1F;
green = (color >> 5) & 0x3F;
blue = color & 0x1F;
```

NOW it works

- let's extract green

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	1	1	0	0	0	0	0	1	0	0	1	1

shift right by 5 and...

0	0	0	0	0	1	0	1	1	1	1	0	0	0	0	0

and then **AND** with **0x3F**...

0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0

Can't you AND *then* shift?

- sure, but…

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	1	1	0	0	0	0	0	1	0	0	1	1

AND with **0x7E0 (!)**…

0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0

shift right by 5…

0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0

where did I get 0x7E0??

it's 0x3F << 5.

I feel like that's uglier.

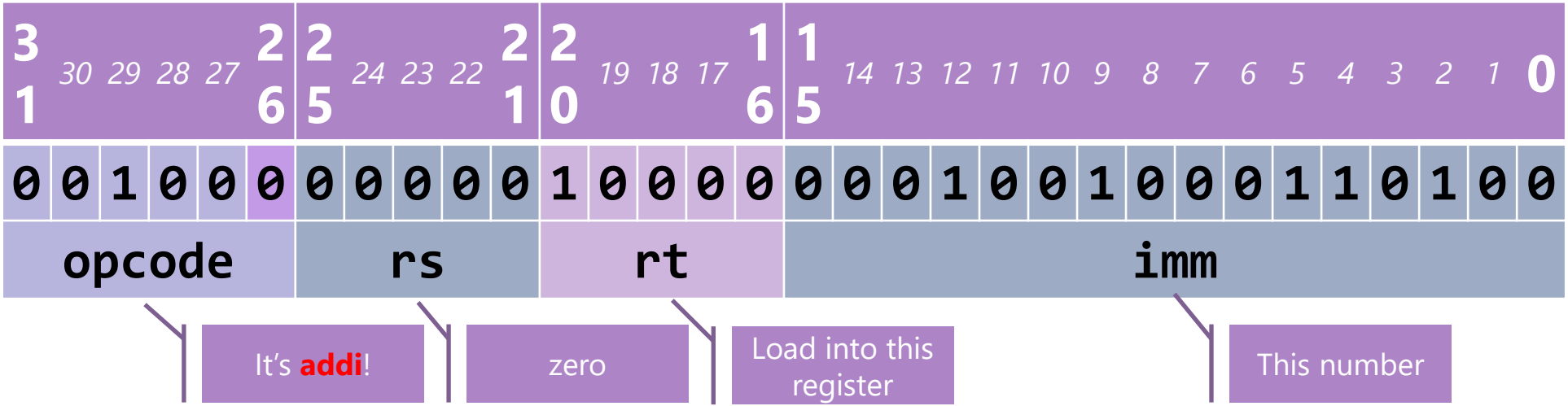
Experience shows me it leads to more mistakes

So don't do this: Keep the mask aligned to the right!!!

Exercise!

- This bitfield represents the following MIPS instruction:

  addi s0, zero, 0x1234



- Using these operations: **<<**, **>>**, **&**, **|**, **~** write expressions to extract the value of each field
  - E.g.: (**field** **<<** 3)**|**(~0xFFFF)